Min–Min Seeding and Bottleneck Repair for Metaheuristic Cloud Task Scheduling: A Matched Evaluation Against GA–SSA-Style Baselines

Group 38, Cloud Computing

***Abstract*—Scheduling independent tasks on heterogeneous virtual machines (VMs) is a combinatorial problem whose quality directly affects batch completion time. Hybrid metaheuristics that combine a genetic algorithm (GA) with the salp swarm algorithm (SSA) have been proposed for this problem, but their reported gains come from small instances and different baselines. We implement a discrete GA-based scheduler, SA-DGSSA-BR, which adds a Min–Min initial solution, a bottleneck repair move that relocates tasks from the most loaded VM, and an adaptive mutation rule. We compare it with matched, simplified adaptations of GA–SSA, DECSO-style and EcoTaskOpt-style search, plus Min–Min, Round Robin and two ablations, in CloudSim Plus 8.5.7. Across 3,000 method runs on 300 paired instances covering 10 to 500 tasks, two task-length distributions and 30 seeds per cell, every metaheuristic receives the same budget of 1,500 fitness evaluations. At 50–500 tasks, SA-DGSSA-BR reduces mean paired makespan by 8.15% relative to our GA–SSA adaptation (95% bootstrap interval 7.69–8.62%) and by 9.91% relative to a corrected discrete SSA baseline (9.39–10.41%). Bottleneck repair accounts for most of the gain (6.52%, 5.93–7.13%), whereas the adaptive mutation rule shows no measurable benefit. The SSA-style copying step shows no meaningful improvement over plain GA in this model. We do not claim superiority over the original published implementations.**

***Index Terms*—cloud computing, task scheduling, metaheuristics, genetic algorithm, salp swarm algorithm, CloudSim Plus, makespan.**

I. Introduction

Cloud providers execute batches of independent tasks on virtual machines that differ in speed. A poor assignment overloads some VMs while others sit idle, which increases the time until the whole batch completes (the makespan). The general heterogeneous task-mapping problem is NP-complete; static heuristics and metaheuristics therefore provide practical approximations [1].

A 2025 study combined a genetic algorithm with the salp swarm algorithm (GA–SSA) for this problem [2]. Its reported experiment uses 10 tasks and 5 VMs. This paper asks a narrower question: when every method receives the same workload, the same VM configuration and the same search budget, which search components actually reduce makespan and deadline lateness, and how large is the effect at larger scales?

The paper makes three contributions:

1) A matched evaluation framework in CloudSim Plus 8.5.7, with paired instances, a common fitness-evaluation budget, and run-level outputs supplied with the code.
2) An ablation showing that bottleneck repair accounts for most of the improvement of SA-DGSSA-BR over its GA-based variants, and that the adaptive mutation rule contributes nothing measurable.
3) A documented correction of a budget-accounting error in our SSA-style baseline, and evidence that the SSA-style copying step does not improve on plain GA in this batch model.

Our baselines are simplified adaptations of the main search ideas of the cited methods under one common batch model. They are not reproductions of the original programs.

II. Related Work

Braun *et al.* evaluated eleven static mapping heuristics for independent tasks under common assumptions and found Min–Min competitive in several settings [1]. It is a useful low-cost reference point here. The original salp swarm algorithm (SSA) is a continuous optimizer, so applying its leader–follower idea to discrete task-to-VM assignments requires an explicit adaptation [3].

Three recent cloud scheduling papers motivate the matched comparison. Yousef *et al.* combined genetic and SSA search and evaluated ten tasks on five VMs [2]. Gunduz *et al.* proposed DECSO, joining differential evolution and cat swarm optimization and studying makespan, utilization and migration [4]. Sanaj *et al.* proposed EcoTaskOpt, an ant-colony and particle-swarm hybrid with energy and thermal objectives [5]. Their workloads and objectives differ from ours. The corresponding baselines in this paper are simplified, clearly labelled adaptations of their search ideas, not faithful reproductions or direct numerical comparisons with published results.

III. Problem Formulation

There are n independent, non-preemptive tasks and m VMs. Task i has length L_i in million instructions (MI) and soft deadline d_i in seconds. VM j has speed s_j in MIPS. All tasks arrive at time zero, and each VM runs its assigned tasks serially in earliest-deadline-first (EDF) order.

Let $x_{ij} = 1$ if task i is assigned to VM j and 0 otherwise, with $\sum_j x_{ij} = 1$ for every task. Define:

- execution time $p_{ij} = L_i / s_j$;
- busy time $B_j = \sum_i x_{ij}\, p_{ij}$;
- analytical makespan $M = \max_j B_j$;
- normalized positive tardiness $T = \frac{1}{n} \sum_i \max(0, C_i - d_i)/d_i$, where C_i is the EDF completion time;

Algorithm 1: SA-DGSSA-BR for one instance
Input: tasks, VMs, budget $E = 1500$; **Output:** best mapping
1. Set $P \leftarrow$ {Min–Min map, 29 random maps}; score P.
2. While evaluations $< E$:
   retain two elites; select parents by 3-way tournament;
   uniform-crossover, mutate, and score new maps;
   copy assignments from a leader or elite and keep improvements;
   if budget remains, move a long task from the busiest VM
   to another VM if this lowers the complete objective F;
   raise mutation rate after search stagnation.
3. Return the lowest-F mapping encountered.

Fig. 1. Summary pseudocode. Every call to the objective counts against E.

- load imbalance $I = \mathrm{std}(B_1, \ldots, B_m)/\,\mathrm{mean}(B_1, \ldots, B_m)$.

The search minimizes

$$F = 0.50\,\frac{M}{M_0} + 0.20\,I + 0.30\,T, \tag{1}$$

where M_0 is the Min–Min makespan on the same instance. The weights were fixed a priori and not tuned. The search uses the analytical model; reported makespan, tardiness and miss rate use CloudSim Plus finish times.

IV. Proposed Method: SA-DGSSA-BR

SA-DGSSA-BR is a discrete GA with three components:

1) **Initialization.** One Min–Min schedule and 29 random schedules form a population of 30.
2) **Genetic operators.** Tournament selection (size 3), uniform crossover, task-to-VM mutation, and two elite schedules retained each generation.
3) **Discrete SSA-inspired copying.** Each non-elite schedule copies VM assignments from its leader or from the elite. This is a discrete adaptation for integer VM assignments, not the original continuous SSA update.

Two further mechanisms are added. *Adaptive mutation* raises the mutation probability from 0.05 to at most 0.30 as the best score stalls. *Bottleneck repair* identifies the VM with the highest busy time, tests up to five of its longest tasks on every other VM, and keeps the best move only if the full score F improves. Every metaheuristic uses exactly 1,500 calls to the scoring function, including repair evaluations.

A. Baselines

- **GA–SSA adaptation.** A GA with tournament size 3, two-point crossover, mutation 0.10 and two elites, with no Min–Min seed, followed by the same discrete SSA-style copying step. This is our reconstruction, not the authors' implementation.
- **DECSO-style adaptation.** Discrete differential-evolution moves with cat-seeking and tracing-style perturbations. Migration and the authors' full objective are omitted.
- **EcoTaskOpt-style adaptation.** Discrete ant-colony construction with particle-swarm personal and global copying. Energy, thermal, network and scaling components are omitted.
- **DiscreteSSA.** The SSA-style copying step alone, without a GA.
- **Min–Min, Round Robin and plain GA.** Deterministic heuristics and a GA without the SSA step.
- **NoRepair and NoAdapt.** SA-DGSSA-BR with bottleneck repair removed, and with fixed mutation 0.10 in place of the adaptive rule.

B. DiscreteSSA budget correction

The supplied DiscreteSSA implementation applies leader–follower copying without first generating unused GA offspring. Its reported results therefore use the full 1,500-evaluation budget for the operations it performs. This matters because evaluating and discarding offspring would give the method fewer effective search steps. Only the corrected DiscreteSSA results are analyzed here.

V. Experimental Setup

Workloads. Task counts are 10, 50, 100, 200 and 500, with 5, 10, 10, 20 and 20 VMs respectively. Uniform tasks have lengths of 10–100 s on an average-speed VM. Heavy-tailed tasks are drawn from a lognormal-type distribution with a 10 s floor and a 350 s cap, so the tail is truncated. VM speeds are uniform on 800–1,200 MIPS.

Deadlines. Each task deadline equals the fluid lower bound on makespan (total work divided by total speed) multiplied by a factor drawn uniformly from 0.25–1.15. These are synthetic soft deadlines for stress testing. Some are infeasible even for an optimal schedule, and they are not service-level agreements.

Pairing. For each workload, task count and seed, every method runs on the identical instance. Within each seed, method order is shuffled to reduce order effects.

Budget. All metaheuristics use 1,500 scoring calls. Min–Min and Round Robin use none.

Simulation. Each schedule is replayed in CloudSim Plus 8.5.7 [6] on Java 17, with one VM per processing element and space-shared scheduling. A self-test checks a four-second hand-calculated example. Across all runs, the largest difference between CloudSim and analytical makespan is 0.216% of the analytical value.

Statistics. The primary statistic is the mean paired percentage improvement, $100\,(\text{baseline} - \text{proposal})/\text{baseline}$, computed per seed and averaged. We report 95% percentile bootstrap intervals from 10,000 resamples of the seed-level differences. Pooled 50–500-task intervals resample all 240 paired cases.

Timing. Optimizer runtime is measured inside the Java process around each optimizer call, excluding CloudSim. Each run is a single execution on one machine, without warm-up repetitions.

VI. Results and Discussion

A. Makespan against matched baselines

Table I reports the mean paired improvement of SA-DGSSA-BR over each comparator. Positive values favour SA-DGSSA-BR. Intervals are 95% bootstrap intervals over 30 paired seeds per cell; the pooled 10-task and 50–500-task rows use 60 and 240 cases.

TABLE I
MEAN PAIRED MAKESPAN IMPROVEMENT OF SA-DGSSA-BR (%), WITH 95% BOOTSTRAP INTERVALS.

Workload	Tasks	vs GA–SSA	vs DECSO	vs EcoTaskOpt	vs Min–Min
Uniform	10	0.41 (-1.14, 2.00)	-0.08 (-1.16, 0.96)	-0.13 (-1.51, 1.11)	21.36 (19.14, 23.69)
Uniform	50	8.14 (7.17, 8.99)	9.94 (9.18, 10.71)	8.97 (8.22, 9.74)	10.54 (9.77, 11.35)
Uniform	100	3.82 (3.21, 4.44)	4.59 (3.95, 5.21)	4.68 (4.13, 5.23)	2.84 (2.36, 3.31)
Uniform	200	11.66 (11.01, 12.30)	13.03 (12.26, 13.79)	11.89 (11.24, 12.46)	2.89 (2.57, 3.22)
Uniform	500	8.37 (7.79, 8.91)	9.96 (9.34, 10.54)	8.43 (8.00, 8.85)	0.41 (0.31, 0.51)
Heavy	10	0.02 (-1.38, 1.54)	-0.29 (-1.69, 0.91)	0.78 (-0.59, 2.27)	26.52 (24.30, 28.76)
Heavy	50	6.81 (5.09, 8.65)	9.22 (7.57, 10.83)	7.03 (5.80, 8.21)	31.55 (29.15, 33.89)
Heavy	100	6.02 (5.30, 6.75)	7.92 (7.19, 8.63)	7.42 (6.64, 8.15)	23.34 (20.65, 26.20)
Heavy	200	10.26 (8.92, 11.58)	15.32 (14.43, 16.13)	11.38 (10.13, 12.62)	27.08 (25.18, 28.87)
Heavy	500	10.11 (9.40, 10.79)	12.88 (12.26, 13.49)	10.02 (9.38, 10.61)	15.67 (14.73, 16.54)
All	10	0.21 (-0.84, 1.28)	-0.19 (-1.08, 0.62)	0.32 (-0.67, 1.30)	23.94 (22.25, 25.73)
All	50–500	8.15 (7.69, 8.62)	10.36 (9.85, 10.85)	8.73 (8.31, 9.12)	14.29 (12.79, 15.83)

TABLE II
MEAN PAIRED MAKESPAN IMPROVEMENT OF SA-DGSSA-BR OVER ITS ABLATIONS, 50–500 TASKS (240 PAIRED CASES).

Comparison	Mean (%)	95% interval	Ties	Wins / losses
vs NoRepair	6.52	(5.93, 7.13)	3	234 / 3
vs NoAdapt	0.19	(-0.13, 0.53)	143	48 / 49

Across 50–500 tasks, SA-DGSSA-BR has lower makespan than the GA–SSA adaptation in 237 of 240 paired cases and loses in 3. It is lower than the DECSO-style adaptation in 238 cases and the EcoTaskOpt-style adaptation in 238 cases. Against Min–Min it is at least as good in all 240 cases, with 237 strict improvements and 3 ties.

At 10 tasks, comparisons with the three metaheuristic adaptations are not distinguishable from zero; the improvement over Min–Min is positive. The DECSO-style adaptation is marginally better on average (-0.19%), and the advantage over GA–SSA is inconclusive (0.21%).

Against the corrected DiscreteSSA baseline, the improvement at 50–500 tasks is 9.91% (interval 9.39–10.41%), with 236 wins, 1 tie and 3 losses.

Figure 2 shows median makespan by task count, and Figure 3 shows the paired improvements with bootstrap intervals.

B. Relation to plain GA

The SSA-style step shows no meaningful gain in this matched comparison. Across the 240 cases at 50–500 tasks, the GA–SSA adaptation is 0.03% better than plain GA on average and wins in 126 cases. The corrected DiscreteSSA baseline is 1.94% worse than plain GA on average and wins in only 61 of 240 cases. These are point estimates computed from the supplied corrected run-level CSV, without intervals.

Together with the ablation below, the evidence identifies bottleneck repair as a major contributor. The separate contribution of Min–Min seeding remains untested.

C. Ablation

Bottleneck repair accounts for most of the improvement. The adaptive mutation rule shows no measurable benefit: its interval includes zero, and 143 of 240 cases yield identical schedules under both settings. NoRepair retains the Min–Min seed and the adaptive mutation, so Table II measures the effect of repair given the seed; it does not measure the seed alone. A variant without the Min–Min seed was not tested, so the separate contributions of seeding and repair are not established.

D. Comparison with Min–Min and runtime

At 500 uniform tasks, the median makespans are 1,432.04 s for SA-DGSSA-BR and 1,433.98 s for Min–Min, a difference of 0.14%; the paired comparison at this size is 0.41% (interval 0.31–0.51%). Median optimizer time at 500 heavy tasks is 73.8 ms for SA-DGSSA-BR and 4.7 ms for Min–Min. Min–Min is effectively free by comparison, so the trade-off is between a small makespan gain and additional scheduling computation. These timings come from one machine and one JVM and are not general speed claims.

E. Deadlines and load balance

At 100 uniform tasks, SA-DGSSA-BR and Min–Min have the same deadline miss rate (14.5%), so the makespan gain does not reduce misses there. At 100 heavy tasks, SA-DGSSA-BR has median tardiness 0.022, miss rate 14.5% and load CV 0.021, compared with 0.054, 21.0% and 0.168 for Min–Min. Because the synthetic deadlines lie close to the fluid bound, tardiness largely tracks makespan, so we treat deadline results as secondary.

F. Scalability

Scaling results mix two changes: task count and VM count. Tasks per VM are 2, 5, 10, 10 and 25 across the five sizes. A fixed-VM study has not been run, so scalability conclusions refer to the joint change.

G. Statistical reliability

For each 50–500-task cell in Table I, the reported 95% interval excludes zero; the Min–Min interval at uniform 500 tasks is positive but small (0.31–0.51%). At 10 tasks, the three metaheuristic-adaptation comparisons are inconclusive, while Min–Min is outperformed. No correction for multiple

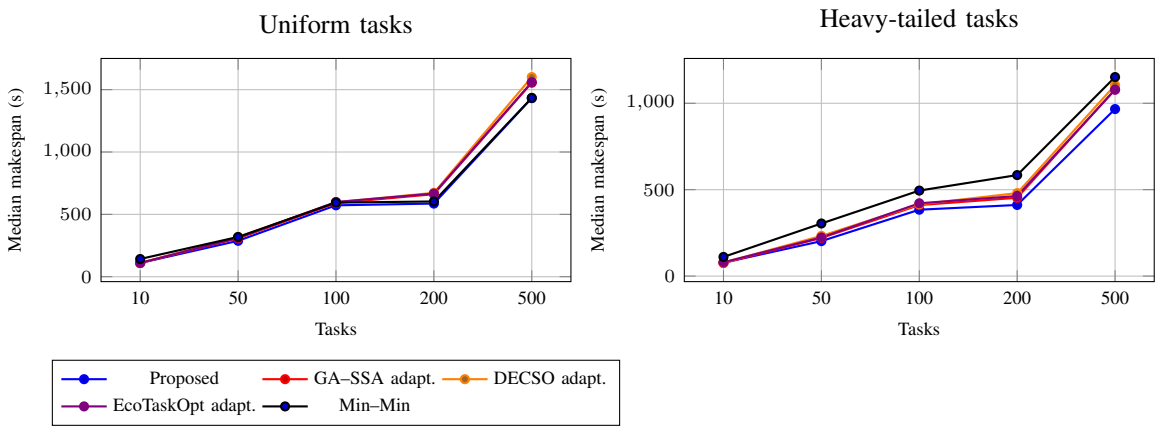


Fig. 2. Median CloudSim makespan across 30 paired seeds at each task count. These curves use the corrected run-level data supplied with the paper.

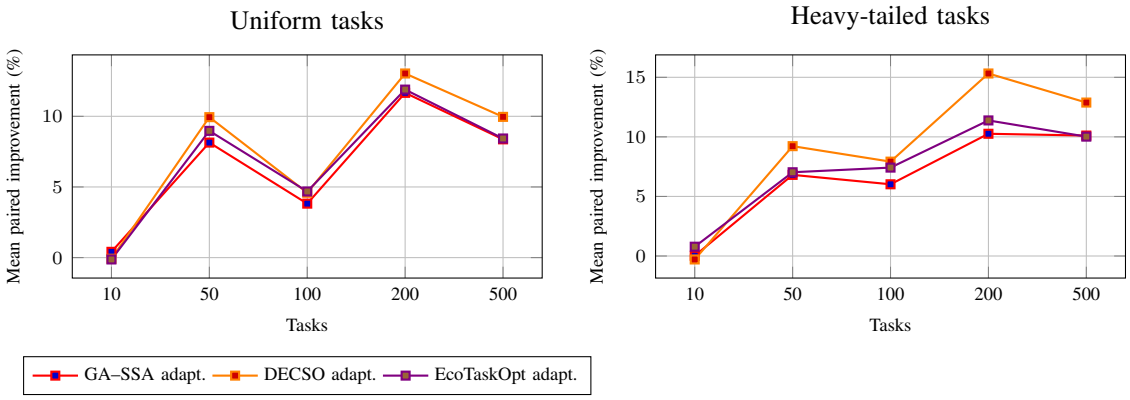


Fig. 3. Mean paired makespan improvement against the three paper-inspired adaptations. Positive values favor the proposal. Table I gives 95% bootstrap intervals.

comparisons was applied. The intervals describe variation across synthetic instances, not uncertainty about real cloud workloads.

VII. Limitations

1) **Synthetic workloads and deadlines.** No real cloud trace was used, and deadlines were generated by formula.
2) **Simplified baselines.** The GA–SSA, DECSO and EcoTaskOpt methods are documented adaptations under our batch model. The published results cited in Section II are not reproductions and are not directly comparable with ours.
3) **Incomplete factorial ablation.** The Min–Min seed was not tested alone, and the SSA step was not switched off within the full method.
4) **Adaptive mutation.** The rule is part of the algorithm as implemented and reported, but the ablation shows it has no measurable effect. We recommend removing it in future work.
5) **Budget accounting.** Budgets are equal in evaluations, not wall-clock time. Repair spends evaluations that plain GA would spend on additional generations.
6) **Budget correction.** Only the corrected DiscreteSSA run is supplied and reported; this package does not establish the numerical effect of the earlier implementation error.
7) **Timing.** All runtimes come from one machine and one JVM, without warm-up.
8) **No energy or cost model.** We make no claims about energy use or monetary cost.
9) **Fixed weights.** The objective weights (0.50, 0.20, 0.30) were set a priori and not tuned or tested for sensitivity.

VIII. Conclusion and Future Work

On matched synthetic batch workloads, bottleneck repair applied to a Min–Min-seeded, GA-based scheduler reduces makespan compared with a GA–SSA-style adaptation, a corrected discrete SSA baseline, and DECSO-style and EcoTaskOpt-style adaptations. The ablation identifies repair as a major contributor; the SSA-style step shows no meaningful gain over plain GA, and the adaptive mutation rule shows no measurable benefit. Comparisons with the three metaheuristic adaptations at 10 tasks are inconclusive.

Future work should (i) isolate the Min–Min seed with a factorial ablation, (ii) evaluate on a real or realistic task trace, (iii) run a fixed-VM scalability study with nested task sets, (iv) perform weight sensitivity analysis on held-out seeds,

and (v) reproduce the published baselines faithfully if their implementations become available.

REFERENCES

[1] T. D. Braun *et al.*, "A comparison of eleven static heuristics for mapping a class of independent tasks onto heterogeneous distributed computing systems," *J. Parallel Distrib. Comput.*, vol. 61, no. 6, pp. 810–837, 2001, doi: 10.1006/jpdc.2000.1714.

[2] M. Yousef, N. Hassouneh, and A. Sharieh, "Hybrid metaheuristic-based cloud task scheduling using genetic algorithm and salp swarm algorithm," in *Proc. Int. Conf. New Trends Comput. Sci. (ICTCS)*, 2025, pp. 406–412, doi: 10.1109/ICTCS65341.2025.10989323.

[3] S. Mirjalili, A. H. Gandomi, S. Z. Mirjalili, S. Saremi, H. Faris, and S. M. Mirjalili, "Salp swarm algorithm: A bio-inspired optimizer for engineering design problems," *Adv. Eng. Softw.*, vol. 114, pp. 163–191, 2017, doi: 10.1016/j.advengsoft.2017.07.002.

[4] A. Gunduz, S. Senan, and Z. Gurkas-Aydin, "A hybrid DECSO algorithm for efficient multi objective task scheduling in cloud computing environments," *Cluster Comput.*, vol. 28, art. no. 848, 2025, doi: 10.1007/s10586-025-05586-5.

[5] M. S. Sanaj *et al.*, "Energy-efficient task scheduling in cloud computing with EcoTaskOpt: A hybrid ant colony and particle swarm optimization approach," *Int. J. Comput. Intell. Syst.*, vol. 19, art. no. 64, 2026, doi: 10.1007/s44196-025-01095-w.

[6] M. C. Silva Filho, R. L. Oliveira, C. C. Monteiro, P. R. M. Inácio, and M. M. Freire, "CloudSim Plus: A cloud computing simulation framework pursuing software engineering principles for improved modularity, extensibility and correctness," in *Proc. IFIP/IEEE Symp. Integr. Netw. Serv. Manag. (IM)*, 2017, doi: 10.23919/INM.2017.7987304.